Diffusion Models: From Adding Noise to Generating Images

CS420 Seminar [S06]

Vong Chi Van (24125049) Le Dat Phuc An (24125052)

University of Science, VNU-HCM

https://github.com/ChiVan222/AI_Seminar_Diffusion

Part I (Slides 2–6): Vong Chi Van Part II (Slides 7–11): Le Dat Phuc An

Problem & Guiding Question

Goal of generative modeling: learn the data distribution $p(\mathbf{x})$ so we can sample *new* images.

Earlier approaches each give something up:

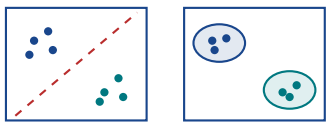
- **VAEs** are fast, but samples are blurry.
- **GANs** are sharp, but training is unstable and they miss modes.
- **Autoregressive models** are exact, but generate one pixel at a time.

How can learning to remove noise step by step let a network generate images, and why does moving this process into a latent space make it much cheaper?

Papers: DDPM, Ho et al., NeurIPS 2020 [1] (required) · Latent Diffusion, Rombach et al., CVPR 2022 [2]
Our experiments: re-implemented on MNIST, trained on one NVIDIA T4 GPU.

Task 1 — Discriminative vs. Generative Models

- **Discriminative** $P(y \mid \mathbf{x})$: learns a *boundary* between classes. It classifies, but it cannot create data.
- **Generative** $p(\mathbf{x})$: learns *where data lives*, so it can sample new $\mathbf{x}$.



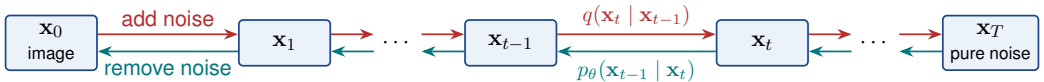
boundary $P(y \mid \mathbf{x})$ density $p(\mathbf{x})$

Qualitative comparison

	Quality	Diversity	Speed
VAE [3]	blurry	good	fast
GAN [4]	sharp	collapse	fast
PixelCNN [5]	sharp	good	very slow
Diffusion [1]	**sharp**	**good**	slow

Diffusion gives up *speed* to get both quality and diversity. LDM is designed to recover that speed.

Task 2 — Forward Noising and Reverse Denoising



Forward process (fixed, no learning)

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\big(\sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t \mathbf{I}\big)$$

Small noise $\beta_t \in (0,1)$ is added at each step. Any step can be reached directly:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \quad \bar{\alpha}_t = \textstyle\prod_{s \le t}(1-\beta_s)$$

Reverse process (learned)

- A U-Net $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$ predicts the noise $\boldsymbol{\epsilon}$ that was added.
- Training objective [1]:

$$L_{\text{simple}} = \mathbb{E}_{t,\mathbf{x}_0,\boldsymbol{\epsilon}} \big\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \big\|^2$$

Equivalent to learning the score $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ [6].

Task 2 — Training and Sampling Algorithms

Algorithm 1: Training

1. Take a real image $\mathbf{x}_0$
2. Pick a random step $t \in \{1, \dots, T\}$ and noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
3. Make $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}$
4. Update θ to minimize $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$

Each step is a simple regression with **one** network pass, so training is stable.

Algorithm 2: Sampling

1. Start from pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2. For $t = T, \dots, 1$, remove the predicted noise:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$$

3. Return $\mathbf{x}_0$

Needs T **sequential** network passes ($T = 1000$ in [1]). This is why sampling is slow.

Task 3 — A 1D Worked Example

Symbols

$\mathbf{x}_0$	clean data, scaled to $[-1, 1]$
$\mathbf{x}_t$	noisy data at step t
β_t	noise variance, $(0, 1)$
$\bar{\alpha}_t$	signal kept, from 1 to 0
$\boldsymbol{\epsilon}$	Gaussian noise $\mathcal{N}(\mathbf{0}, \mathbf{I})$
$\boldsymbol{\epsilon}_\theta$	network's noise guess

Given: $x_0 = 0.80$, $\beta_1 = 0.19$, $\epsilon = -0.50$
$\Rightarrow\ \sqrt{\bar{\alpha}_1} = 0.90,\ \sqrt{1 - \bar{\alpha}_1} = 0.436$

1. Forward step

$$x_1 = 0.90(0.80) + 0.436(-0.50) = \mathbf{0.502}$$

The mean shrinks toward 0 (0.72) while noise is added, which keeps the total variance at 1.

2. Loss (suppose the network predicts $\epsilon_\theta = -0.30$)

$$L = (-0.50 - (-0.30))^2 = \mathbf{0.04}$$

3. Reverse step

$$\hat{x}_0 = \tfrac{1}{0.90}\big(0.502 + 0.436 \times 0.30\big) = \mathbf{0.703}$$

With a perfect guess ($\epsilon_\theta = -0.50$) we recover exactly $\hat{x}_0 = 0.80$.

Tasks 1 & 4 — Experiments on MNIST

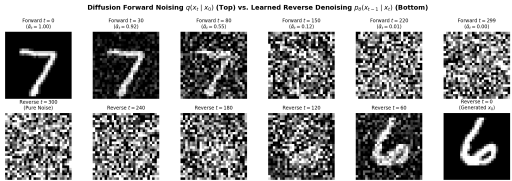
Same data, four model families

Model	Steps	Time / 64	FID ↓
VAE	1	6.5 ms	82.3
PixelCNN	784	4.29 s	64.1
DCGAN	1	11.4 ms	44.1
DDPM	300	3.93 s	**16.9**

FID is computed with our MNIST classifier (98.85% test accuracy).

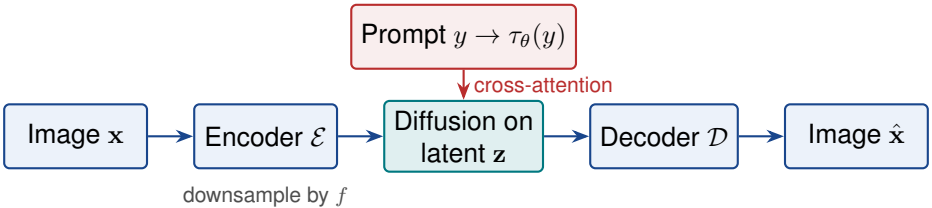
- DDPM gives the **best quality**.
- It is about $\mathbf{350}\times$ **slower** than the GAN.

Task 4: noising and denoising



Top: the digit fades into noise.
Bottom: a digit emerges from pure noise.

Task 5 — Latent Diffusion Models [2]



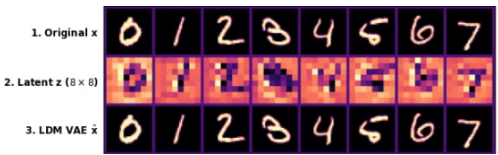
1. **Compress:** an autoencoder maps the image to a small latent grid and discards fine pixel detail we cannot perceive.
2. **Diffuse in latent space:** the same DDPM runs on $\mathbf{z}$. Attention cost drops by about f^4.
3. **Condition:** prompt tokens act as keys/values in cross-attention, $\text{softmax}\big(\mathbf{Q}\mathbf{K}^\top/\sqrt{d}\big)\mathbf{V}$.

Classifier-free guidance (scale α) pushes samples toward the prompt:

$$\tilde{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, \varnothing) + \alpha\left(\boldsymbol{\epsilon}_\theta(\mathbf{z}_t, y) - \boldsymbol{\epsilon}_\theta(\mathbf{z}_t, \varnothing)\right)$$

Task 5 — Our Latent Diffusion Results

Stage 1: VAE, $32\times32 \to 8\times8\times4$



- Reconstruction **FID 0.18**, PSNR 33.7 dB
- 99.8% of digits keep their label

The small latent keeps almost all of the digit content.

Stage 2: prompt = digit label "0"–"9"

CFG α	Prompt acc.	FID ↓	Saturated
1.0	94.0%	**28.1**	4.5%
3.0	**100%**	71.1	5.6%
7.5	99.5%	94.3	7.5%
15.0	99.5%	142.3	10.6%

- Guidance makes the model **follow the prompt**.
- Too much guidance hurts **diversity** and causes **oversaturation**.

Task 6 — Pixel-Space vs. Latent-Space Diffusion

Same U-Net design, same data and training setup; only the space changes (T4 GPU).

	Pixel diffusion (DDPM)	**Latent diffusion (LDM)**
Works on	$32 \times 32 \times 1$ pixels	$8 \times 8 \times 4$ latent
Training time	51.4 s / epoch	**6.8 s / epoch** ($7.6\times$)
Peak memory	1,152 MB	**83 MB** ($13.8\times$ **less**)
Quality	strong at low resolution	strong at high resolution [2]
Conditioning	limited	flexible (cross-attention)
Main weakness	cost grows fast with size	capped by the autoencoder

Takeaway: moving diffusion into a compressed latent space gives large compute savings with little loss in quality.

Limitations & Conclusion

Limitations

- **Slow sampling:** many sequential steps. Faster samplers [7, 8] help but lose some detail.
- **Prompt control:** strong guidance oversaturates; complex prompts are hard.
- **Memorization:** models can copy training images [9].
- **Bias:** web-scale data carries social bias.
- **Our scope:** MNIST on one GPU. Gains should grow at high resolution, but we did not test that.

Conclusion

Diffusion models turn generation into a sequence of easy denoising steps. This gives sharp *and* diverse samples with stable training.

Latent diffusion keeps these benefits but runs in a compact latent space. That makes high-resolution, prompt-controlled generation practical.

References & Q&A

[1] J. Ho, A. Jain, P. Abbeel. Denoising Diffusion Probabilistic Models. *NeurIPS*, 2020.

[2] R. Rombach et al. High-Resolution Image Synthesis with Latent Diffusion Models. *CVPR*, 2022.

[3] D. P. Kingma, M. Welling. Auto-Encoding Variational Bayes. *ICLR*, 2014.

[4] I. Goodfellow et al. Generative Adversarial Nets. *NeurIPS*, 2014.

[5] A. van den Oord et al. Pixel Recurrent Neural Networks. *ICML*, 2016.

[6] Y. Song, S. Ermon. Generative Modeling by Estimating Gradients of the Data Distribution. *NeurIPS*, 2019.

[7] J. Song, C. Meng, S. Ermon. Denoising Diffusion Implicit Models. *ICLR*, 2021.

[8] C. Lu et al. DPM-Solver: A Fast ODE Solver for Diffusion Sampling. *NeurIPS*, 2022.

[9] N. Carlini et al. Extracting Training Data from Diffusion Models. *USENIX Security*, 2023.

Reproducibility

Code, notebooks and figures: github.com/ChiVan222/AI_Seminar_Diffusion

AI use

AI assistants helped with LaTeX formatting and plotting. All derivations and results were checked by the team.

Thank you!

Questions?